

CHAPTER 7: SOFTWARE CONFIGURATION MANAGEMENT

❖ SOFTWARE CONFIGURATION MANAGEMENT

Software Configuration Management (SCM) is the process of systematically managing, organizing, and controlling changes to documents, source code, and other software artifacts throughout the Software Development Life Cycle (SDLC).

“Configuration management is the discipline of managing and controlling change in the evolution of software systems.”

— IEEE Std. 1042-1987

When you build computer software, change happens. And because it happens, you need to manage it effectively. Software configuration management (SCM), also called change management, is a set of activities designed to manage change by identifying the work products that are likely to change, establishing relationships among them, defining mechanisms for managing different versions of these work products, controlling the changes imposed, and auditing and reporting on the changes made.

Software configuration management (SCM) is an umbrella activity that is applied throughout the software process. Because change can occur at any time, SCM activities are developed to identify change, control change, ensure that change is being properly implemented, and report changes to others who may have an interest. The four fundamental sources of change on software products are:

- New business or market conditions dictate changes in product requirements or business rules.
- New stakeholder needs demand modification of data produced by information systems, functionality delivered by products, or services delivered by a computer-based system.
- Reorganization or business growth/downsizing causes changes in project priorities or software engineering team structure.
- Budgetary or scheduling constraints cause a redefinition of the system or product.

Baselines:

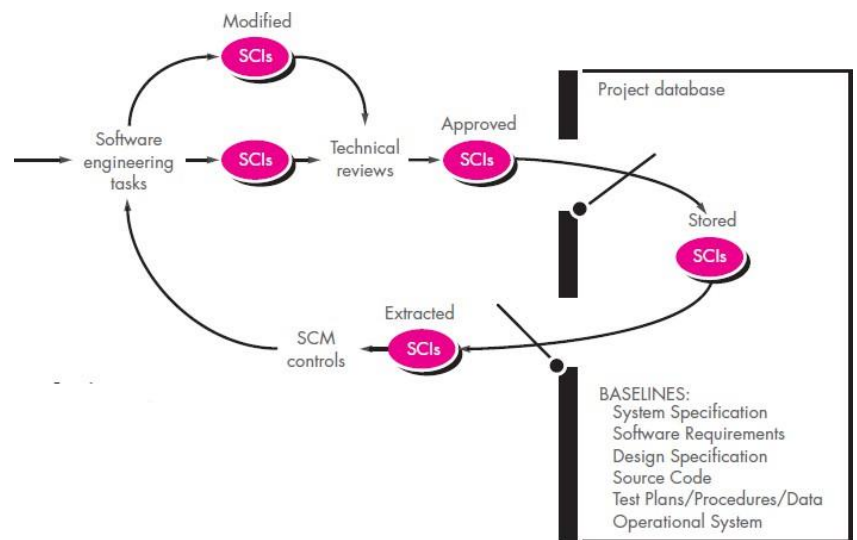
In the context of software engineering, a baseline is a milestone in the development of software. A **baseline** is a formally reviewed and approved version of one or more software configuration items that serves as a reference point for further development and can be modified only through formal change control procedures. A baseline is marked by the delivery of one or more software configuration items that have been approved as a consequence of a technical review. For example, the elements of a design model have been documented and reviewed. Errors are found and corrected. Once all parts of the model have been reviewed, corrected, and then

approved, the design model becomes a baseline. Further changes to the program architecture (documented in the design model) can be made only after each has been evaluated and approved.

Before a software configuration item (SCI) becomes a baseline (or placed in a *project database*, also called a *project library* or *software repository*), change can be made quickly or informally. However, once a baseline is established, we figuratively pass through a swinging one way door.

Now, the formal procedure must be applied to evaluate and verify each change. The procedure to make changes on baseline product will be:

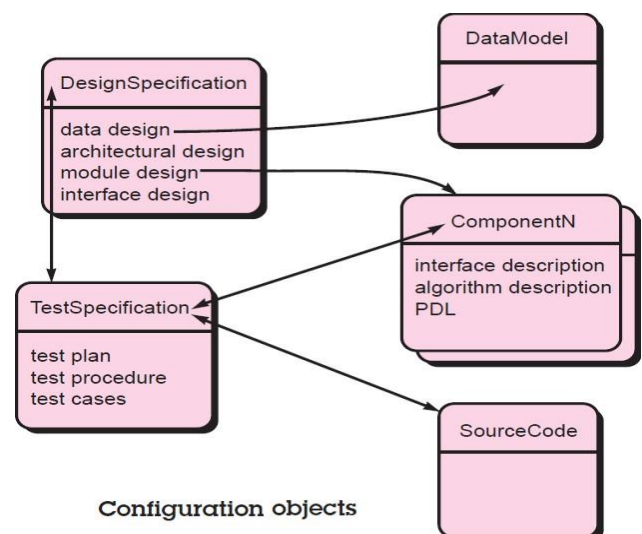
- Send a private copy of module needing modification to change control board (CCB)
- Get permission from the CCB for restoring change module
- After review from CCB, the manager updates the old baseline by restore operation.



Baselined SCIs and the project database

Software Configuration Items (SCI):

An SCI is any document, code, or artifact produced during software development that is placed under configuration management and controlled for changes. The first law of software engineering is “No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle”. The



Configuration objects

change is a fact of life in software development. Customer wants to modify the requirements, developer wants to modify the technical approach, manager wants to modify the project strategy etc.

A configuration object has a name, attributes, and is “connected” to other objects by relationships. Referring to Figure, the configuration objects, Design-Specification, Data-Model, Component-N, Source-Code, and Test-Specification are each defined separately. However, each of the objects is related to the others as shown by the arrows. A curved arrow indicates a compositional relation. That is, Data-Model and Component-N are part of the object Design- Specification. A double-headed straight arrow indicates an interrelationship. If a change were made to the Source-Code object, the interrelationships enable you to determine what other objects (and SCIs) might be affected.

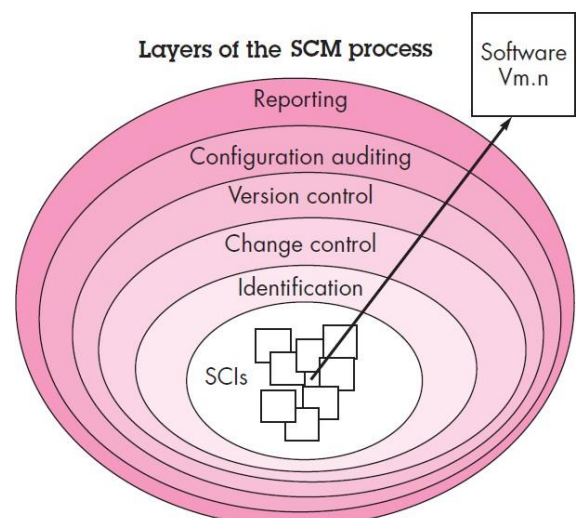
Key objectives of SCM:

- 1. Control the evolution of software systems:** SCM helps to ensure that changes to a software system are properly planned, tested, and integrated into the final product.
- 2. Enable collaboration and coordination:** SCM helps teams to collaborate and coordinate their work, ensuring that changes are properly integrated and that everyone is working from the same version of the software system.
- 3. Provide version control:** SCM provides version control for software systems, enabling teams to manage and track different versions of the system and to revert to earlier versions if necessary.
- 4. Facilitate replication and distribution:** SCM helps to ensure that software systems can be easily replicated and distributed to other environments, such as test, production, and customer sites.

❖ SCM PROCESS

The software configuration management process defines a series of tasks that have four primary objectives:

1. To identify all items that collectively define the software configuration,
2. To manage changes to one or more of these items,



3. To facilitate the construction of different versions of an application, and
4. To ensure that software quality is maintained as the configuration evolves over time.

SCM process:

1. Identification
2. Version Control
3. Change Control
4. Configuration Audit
5. Status Reporting

Explanation:

1. Identification of objects in the software configuration

Identification of objects in the software configuration is the SCM activity in which all software configuration items (SCIs) are systematically selected, defined, and uniquely identified so that they can be effectively managed throughout the software development life cycle. This process involves determining which documents, source code modules, data files, and other software artifacts require configuration control, assigning them unique identifiers, and defining their relationships. Proper identification establishes baselines, enables accurate version and change control, and ensures the integrity and traceability of software products during evolution.

To control and manage software configuration items, each should be separately named and then organized using an object-oriented approach. Two types of objects can be identified: basic objects and aggregate objects.

- **A basic object** is a unit of information that you create during analysis, design, code, or test. For example, a basic object might be a section of a requirements specification, part of a design model, source code for a component, or a suite of test cases that are used to exercise the code.
- **An aggregate object** is a collection of basic objects and other aggregate objects. For example, a Design Specification, which consists of multiple design diagrams, descriptions, and components, is an aggregate object.

Tools like Git and SVN (subversion) help in identification and tracking of software configuration items by giving each file a unique identity, version number, and change history.

Benefits of identification of objects in SCM:

- **Traceability & Accountability**

Identifying objects ensures every artifact has a clear history, showing who made changes and why, which is key for audits and debugging.

- **Controlled Change Management**

Versioning allows multiple versions to coexist, prevents conflicts, and helps manage integrations smoothly.

- **Enhanced Quality & Reliability**

Proper identification supports audits, ensures the correct components are released, and enables automated testing by recognizing software elements, improving overall stability.

- **Improved Collaboration**

With identified objects, all team members share a common understanding, reducing miscommunication.

- **Efficient Builds & Releases**

Linking related objects into baselines ensures consistent builds and releases, avoiding errors from mismatched versions.

- **Reduced System Disruptions**

Understanding dependencies allows controlled changes, reducing unexpected issues and downtime.

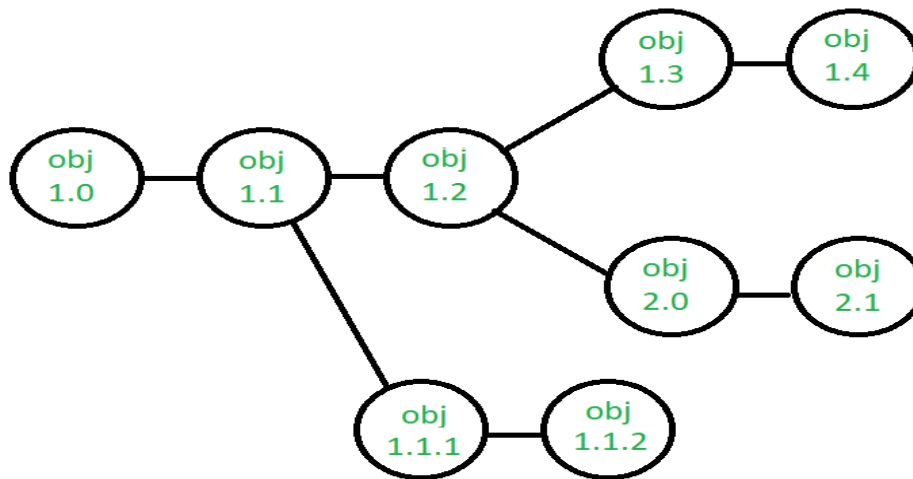
2. Version control

Version control is a systematic process of managing and tracking changes to software artifacts, such as source code, documents, and configuration items, over time.

Version control combines procedures and tools to manage different versions of configuration objects that are created during the software process. A version control system implements or is directly integrated with four major capabilities:

1. A project database (repository) that stores all relevant configuration objects,
2. A version management capability that stores all versions of a configuration object (or enables any version to be constructed using differences from past versions),
3. A make facility that enables you to collect all relevant configuration objects and construct a specific version of the software.

In addition, version control and change control systems often implement an issues tracking (also called bug tracking) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object. Creating versions/specifications of the existing product to build new products with the help of the SCM system. A description of the version is given below:



Suppose after some changes, the version of the configuration object changes from 1.0 to 1.1. Minor corrections and changes result in versions 1.1.1 and 1.1.2, which is followed by a major update that is object 1.2. The development of object 1.0 continues through 1.3 and 1.4, but finally, a noteworthy change to the object results in a new evolutionary path, version 2.0. Both versions are currently supported.

Steps in the Version Control Process

1. Initial Setup

- The development team creates a Git repository to manage the project's code and track changes.
- A baseline version of the software is stored, which serves as the starting point for all future development.

2. Adding a New Feature

- The team decides to implement a new feature, e.g., "User Rating for Deliveries."
- A separate branch (e.g., add-rating) is created to develop the feature without affecting the main branch.

3. Collaboration and Pull Request (PR)

- Once development is complete, another developer reviews the changes on a remote platform such as GitHub.
- A Pull Request (PR) is created to merge the feature branch into the main branch.
- After review and approval, the feature is merged into the main branch, ensuring the main code remains stable.

4. Bug Fix

- Suppose a bug is reported: “Ratings are not saved for certain delivery IDs.”
- A bug-fix branch is created to isolate the fix.
- After testing, the fix is merged into the main branch, and the version history is updated.

5. Handling a Release

- Before releasing version v1.2.0 of the application, the team tags the release in Git.
- Tagging provides an easy reference to a specific version for deployment or rollback purposes.

6. Rolling Back Changes

- If the release introduces unexpected issues or defects, the team can revert to the last stable commit.
- This ensures the software remains reliable while issues are investigated and resolved.

3. Change control

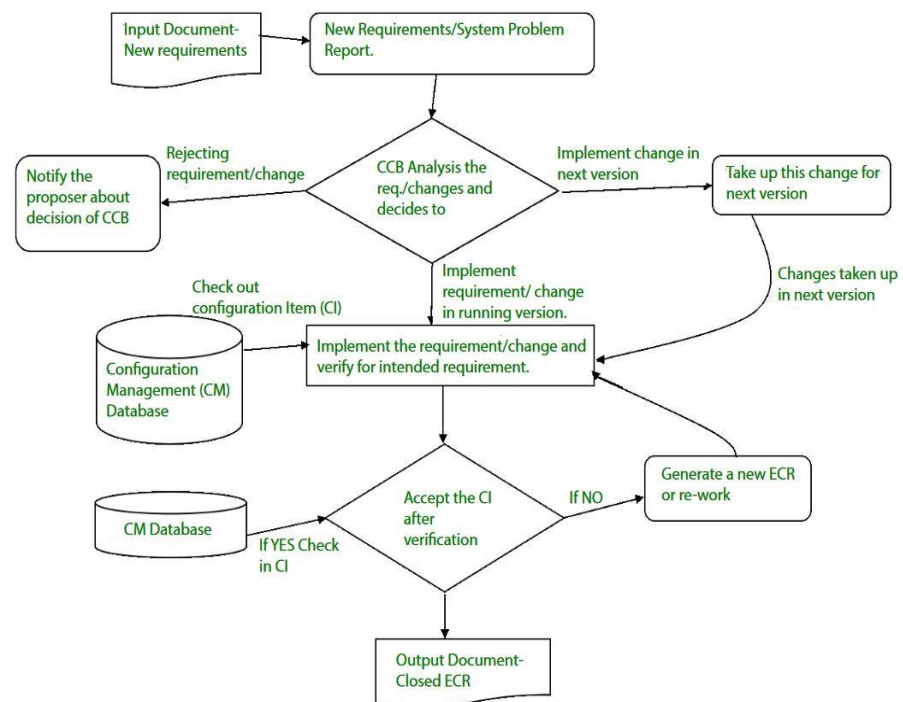
Change Control is the process of managing, evaluating, and implementing changes to software in a controlled and systematic manner to ensure that they do not disrupt the project or introduce risks.

Change control is a procedural activity that ensures quality and consistency as changes are made to a configuration item. The change control process begins with a change request, proceeds with a decision to approve or reject the request, and concludes with a controlled update of the software configuration item that is to be modified.

Change control is a disciplined process that ensures no modification to a software system is made without proper authorization and systematic review. When a stakeholder intends to alter a configuration item, such as source code, design documentation, or a database, a change request is formally submitted. This request is subsequently evaluated by the project manager or a **designated Change Control Board (CCB)** to determine its necessity, feasibility, and potential impact on the system. If the proposed change is approved, it is implemented in a controlled and documented manner. If it is rejected, the existing software configuration remains unaltered. By enforcing this structured procedure, change control prevents unauthorized

modifications, minimizes errors, and reduces confusion, thereby maintaining the stability and reliability of the software while still allowing legitimate improvements.

The change control process is explained in Figure below:



Steps in Change Control Process:

- **Change Request Submission:** The need for a change is identified and formally requested.
- **Impact Analysis:** The technical and business implications of the change are evaluated.
- **Approval or Rejection:** A Change Control Board (CCB) or responsible authority decides whether to approve or reject the change.
- **Implementation:** Approved changes are implemented by the development team.
- **Testing and Validation:** The change is thoroughly tested to ensure it works as intended.
- **Documentation and Deployment:** The change is documented and deployed to production.

4. Configuration audit

A configuration audit in Software Configuration Management (SCM) is a quality assurance process that systematically reviews and verifies all configuration items (CIs) within a software system. A software configuration audit complements the formal technical review of the process and product. It focuses on the technical correctness of the configuration object that has been modified. The audit confirms the completeness, correctness, and consistency of items in the SCM system and tracks action items from the audit to closure. Its purpose is to ensure that:

1. All configuration items are accurate, complete, and consistent.
2. They comply with established standards, baselines, and specifications.
3. The software configuration is properly managed, documented, and controlled according to defined procedures throughout the development lifecycle.

Types of Configuration Audits

1. Functional Configuration Audit (FCA)

The Functional Configuration Audit (FCA) is a formal evaluation conducted to determine whether a software product fulfills its intended functional and performance requirements as defined in its specifications or requirements baseline. This audit focuses primarily on the behavioral aspects of the software and assesses whether all functional objectives, constraints, and acceptance criteria are met.

Example: Checking that a library management system correctly issues and returns books according to specifications.

2. Physical Configuration Audit (PCA)

The Physical Configuration Audit (PCA) is a formal examination of the physical and documentation artifacts associated with the software system to ensure that they conform to design specifications, approved baselines, and organizational standards. The PCA primarily focuses on tangible deliverables, such as code modules, manuals, and configuration records, ensuring that all items are correctly versioned, complete, and properly documented.

Example: Ensuring all modules, manuals, and version-controlled files are present before product delivery.

5. Status reporting

Status Reporting in Software Configuration Management (SCM) is the systematic process of tracking, recording, and communicating the current state of configuration items (CIs), changes, and overall progress of configuration management activities. This process ensures transparency, accountability, and informed decision-making by keeping stakeholders aware of the status of software development and configuration processes, including completed, pending, and upcoming tasks.

Status reporting provides information about each change to those with a need to know. Configuration status reporting (status accounting) is an SCM task that answers the following questions:

- (1) What happened?
- (2) Who did it?
- (3) When did it happen?
- (4) What else will be affected?

Purpose of Status Reporting:

1. **Progress Tracking:** Monitors whether tasks and milestones are being completed according to the project plan.
2. **Transparency:** Keeps stakeholders informed about the current state of the project.
3. **Early Issue Detection:** Highlights risks, delays, or bottlenecks that require corrective action.
4. **Decision Support:** Provides the information needed for management to make informed choices about resources, priorities, and schedules.
5. **Documentation:** Serves as an official record of project performance and changes over time.

Importance of Software Configuration Management (SCM):

1. **Effective Bug Tracking:** Linking code modifications to issues that have been reported, makes bug tracking more effective.
2. **Continuous Deployment and Integration:** SCM combines with continuous processes to automate deployment and testing, resulting in more dependable and timely software delivery.
3. **Risk management:** SCM reduces the risk of introducing critical errors by supporting early detection, controlled implementation, and verification of changes.

4. Support for Big Projects: Source Code Control (SCM) offers an orderly method to handle code modifications for big projects, fostering a well-organized development process.
5. Reproducibility: By recording precise versions of code, libraries, and dependencies, source code versioning (SCM) makes builds repeatable.
6. Parallel Development: SCM facilitates parallel development by enabling several developers to collaborate on various branches at once.

Advantages of SCM:

- Improved productivity and efficiency by reducing the time and effort required to manage software changes.
- Reduced risk of errors and defects by ensuring that all changes were properly tested and validated.
- Increased collaboration and communication among team members by providing a central repository for software artifacts.
- Improved quality and stability of software systems by ensuring that all changes are properly controlled and managed.

Disadvantages of SCM:

- Increased complexity and overhead, particularly in large software systems.
- Difficulty in managing dependencies and ensuring that all changes are properly integrated.
- Potential for conflicts and delays, particularly in large development teams with multiple contributors.

❖ SCM STANDARDS

A Software Configuration Management (SCM) standard specifies the policies, procedures, and responsibilities for managing configuration items (CIs), controlling changes, maintaining versions, performing audits, and reporting status. Over the years, several SCM standards have been proposed to ensure consistency, quality, and traceability of software artifacts.

SCM standards can be classified into **Military standards**, **ANSI/IEEE standards**, and **ISO /IEC standards**.

ANSI - American National Standards Institute

IEEE- Institute of Electrical and Electronics Engineers Standard

ISO - International Organization for Standardization

IEC - International Electrotechnical Commission Standards

1. Military SCM Standards

Military SCM standards were among the earliest SCM standards and were developed for defense and mission-critical software systems, where strict control, reliability, and traceability are essential.

a) MIL-STD-483

MIL-STD-483 is an early military standard that defines basic requirements for software configuration management. It focuses on identifying software items, controlling changes, maintaining records, and performing configuration audits.

b) DOD-STD-480A

DOD-STD-480A is a military SCM standard that improves and extends MIL-STD-483. It emphasizes SCM planning, baseline control, formal change approval, and traceability of software changes.

c) MIL-STD-1521A

MIL-STD-1521A is a military standard that defines technical reviews and configuration audits. It ensures that software matches approved baselines and contractual requirements.

2. ANSI / IEEE SCM Standards

With the growth of commercial and civilian software systems, ANSI and IEEE introduced general-purpose SCM standards applicable to both military and non-military software projects, as well as to small and large organizations.

a) IEEE Std 828

IEEE Std 828 defines the structure and contents of a Software Configuration Management Plan. It specifies SCM roles, configuration identification, change control, status accounting, and configuration audits.

b) IEEE Std 1042

IEEE Std 1042 provides guidelines and best practices for implementing SCM. It explains practical SCM processes, tool support, and integration of SCM with the software life cycle.

c) IEEE Std 1028

IEEE Std 1028 defines procedures for software reviews and audits. It includes technical reviews, inspections, walkthroughs, and configuration audits to ensure quality and compliance.

3. ISO / IEC SCM-Related Standards

ISO/IEC standards are international standards that support SCM as part of quality management and software process frameworks.

a) ISO/IEC 10007

ISO/IEC 10007 provides guidelines for configuration management for both hardware and software. It covers configuration identification, change control, status accounting, and configuration audits.

b) ISO/IEC 12207

ISO/IEC 12207 defines standard software life cycle processes. It includes SCM as a supporting process to ensure controlled and consistent software evolution.

c) ISO 9001

ISO 9001 is a quality management standard applicable to organizations. It requires document control, configuration control, and change management as part of quality assurance.

Importance of SCM Standards

- Ensure consistency and control of software items.
- Provide structured change management and traceability.
- Improve software quality through reviews and audits.
- Support team coordination and role clarity.
- Ensure compliance with standards and contracts.

